

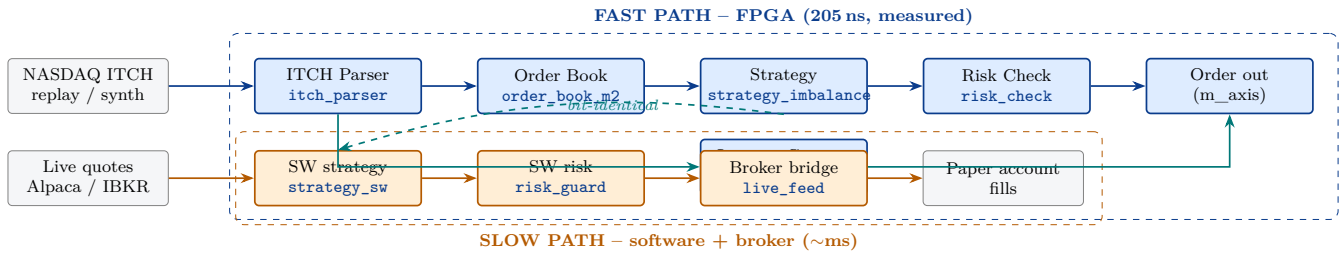
What this project is

A high-frequency trading *tick-to-trade* engine built in **RTL (SystemVerilog)** for an AWS F1 (Xilinx Ultra-Scale+ VU9P) FPGA. It ingests a market-data feed, maintains a live order book, computes a trading signal, runs pre-trade risk checks, and emits an order **entirely in hardware** — a measured **205 ns** (41 clock cycles @ 200 MHz). The same logic is mirrored bit-for-bit in Python both as a verification golden model and as the live software trading path, so the hardware speed can be measured against a real software baseline on real market data.

The thesis: a deeply pipelined spatial dataflow in silicon does in 205 ns what a software stack + broker network does in milliseconds — a $\sim 10,000\text{--}100,000\times$ gap. This document walks the entire flow and explains every file.

1. System Architecture – The Two Paths

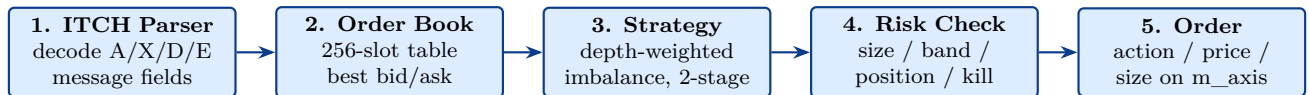
The project is organised around a **dual path**. The **fast (hardware) path** is the FPGA pipeline that is the engineering centrepiece. The **slow (software) path** mirrors the identical logic on a host CPU and routes to a real paper broker, providing both a correctness oracle and the latency baseline to beat.



Why two paths? The FPGA cannot legally sit in a retail broker's critical order path, and raw NASDAQ ITCH is not retail-accessible. So the FPGA's 205 ns is reported as a *measured compute benchmark*; the live software path (which *is* bit-identical to the hardware) trades real paper orders and supplies the apples-to-apples baseline. **Same decisions, two substrates, one latency comparison.**

2. The Hardware Pipeline – Stage by Stage

Each stage is a dedicated circuit; data flows through them like a factory line. Because every stage works *simultaneously* on a different message, the pipeline sustains one message per cycle while any single message clears in 41 cycles.



add/cancel/delete/execute

$$w = \sum_i (N-i) \cdot \text{size}_i$$

72-bit {action, price, size}

How the imbalance signal works. The strategy weights the top $N=4$ book levels by depth ($w = \sum_i (N-i) \cdot \text{size}_i$; the touch counts heaviest) and fires using a division-free cross-multiply:

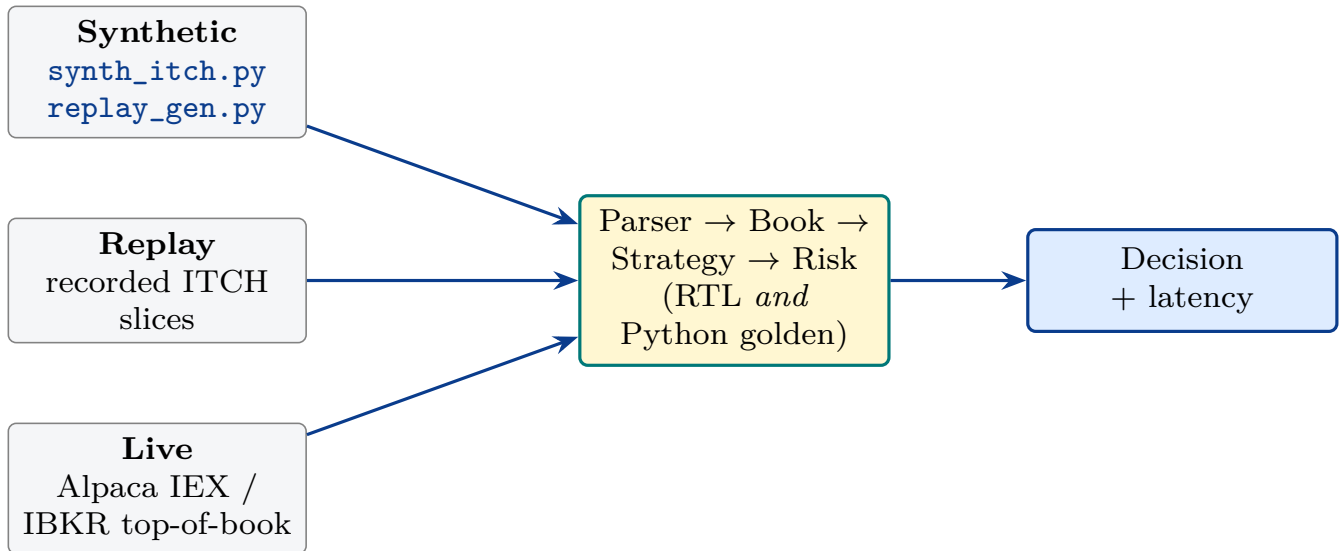
$$\text{BUY if } w_{\text{bid}} \cdot 10 > \text{THRESH} \cdot w_{\text{ask}}, \quad \text{SELL if } w_{\text{ask}} \cdot 10 > \text{THRESH} \cdot w_{\text{bid}}.$$

THRESH=15 means a **1.5×** volume imbalance. The same ratio scales the order lot ($1\times/2\times/3\times$ a base lot, capped). A spread guard, a 2-consecutive-cycle validity gate, and an 8-cycle cooldown prevent spurious fires. **No floating point** anywhere — prices are integers in units of 1/10,000 USD, matching the ITCH wire format.

How latency is measured (in hardware). `latency_counter` latches t_0 at message-start and t_1 when the decision appears, accumulating $\Delta = t_1 - t_0$ into a 64-bucket histogram readable over AXI-Lite (address map in [f1/register_map.md](#)). The headline result: **41 cycles = 205 ns** at 200 MHz.

3. How Data Is Collected – Three Feeds

The exact same parser/book/strategy logic is exercised by three progressively more realistic data sources:

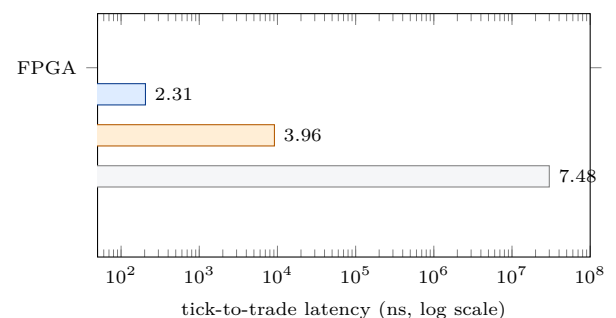


- **Synthetic** — seeded ITCH generators build a realistic book (`sim/synth_itch.py`, `sim/replay_gen.py`) for deterministic, repeatable tests.
- **Replay** — recorded ITCH slices feed the parser to validate book reconstruction against the golden model over large message runs.
- **Live** — `host/live_feed.py` streams real top-of-book quotes (Alpaca free IEX, or IBKR) into the software mirror; the `--simulate` mode uses `host/quote_sim.py` for an offline continuous feed. *Live equities feeds are normalised top-of-book, not raw ITCH* — the FPGA’s ITCH parser is exercised by the synthetic/replay paths, the live path drives the bit-identical software strategy.

4. Why the Hardware Is Faster

A CPU executes the pipeline *temporally*: fetch/decode/execute each instruction of the parser, then the book update, then the strategy, sharing a few ALUs and fighting cache misses and OS jitter. The FPGA lays the same computation out *spatially*: every stage is its own circuit, wired directly to the next, all clocking together. Three compounding effects:

1. **Spatial parallelism** — all 5 stages run every cycle on different messages (one message/cycle throughput).
2. **No instruction overhead** — no fetch/decode, no cache, no OS; logic is the program.
3. **Deterministic** — 41 cycles, every time. No tail latency.



FPGA 205 ns (measured) vs. SW compute $\approx 9 \mu\text{s}$ vs. broker RTT $\approx 30 \text{ ms}$.

Measured comparison (live AAPL soak): FPGA **205 ns** vs. software compute **$\approx 9,149 \text{ ns}$ mean** on the same logic = $\sim 45\times$ on compute alone; once the live *order* path adds network round-trip ($\sim$ tens of ms), the end-to-end gap is ~ 5 orders of magnitude. (See the companion [why_fpga_beats_cpu_parallelism.pdf](#) for the deep dive.)

5. Every File, Explained

RTL – the hardware (rtl/)

File	What it does
<code>itch_parser.sv</code>	Decodes NASDAQ ITCH messages (Add/Cancel/Delete/Execute) into typed fields. Stage 1 of the pipeline.
<code>order_book_m2.sv</code>	M2 order book: 256-slot direct-mapped table, IDLE/RESCAN FSM that recomputes best bid/ask after any cancel/delete/execute.
<code>order_book_top.sv</code>	M1 reference book (top-of-book only); kept for comparison.
<code>strategy_imbalance.sv</code>	Depth-weighted imbalance strategy, 2-stage pipeline, cross-multiply trigger, spread guard, cooldown, lot sizing.
<code>risk_check.sv</code>	Pre-trade risk: max order size, price band (fat-finger), net-position limit, kill switch (Rule 15c3-5 style).
<code>latency_counter.sv</code>	Hardware tick-to-trade timer + 64-bucket histogram, read over AXI-Lite.
<code>tick_to_trade_top.sv</code>	Top level wiring parser→book→strategy→risk→output + latency counter.
<code>multi_symbol_book.sv</code> / <code>multi_symbol_top.sv</code>	4-symbol scaled version (M2): replicated books behind a symbol mux/arbitrer.
<code>top.xdc</code>	Timing/pin constraints (200 MHz clock, false-path AXI-Lite, I/O budget).

Golden models – the Python oracle (sim/golden/)

File	What it does
<code>itch_parser.py</code>	Reference ITCH decoder.
<code>order_book.py</code>	M1 reference book.
<code>order_book_m2.py</code>	Faithful M2 RESCAN book (256-slot, ref-aliasing, top-4 cache) — the book oracle.
<code>strategy_imbalance.py</code>	Faithful depth-weighted strategy mirror — the strategy oracle.

Simulation / verification (sim/)

File(s)	What it does
<code>tb_itch_parser.py</code> , <code>tb_order_book.py</code> , <code>tb_order_book_m2.py</code> , <code>tb_strategy_imbalance.py</code> , <code>tb_risk_check.py</code> , <code>tb_latency_counter.py</code> , <code>tb_tick_to_trade.py</code>	cocotb unit + integration testbenches: drive each RTL module in QuestaSim and assert its outputs match the golden models.
<code>tb_multi_symbol_book.py</code> , <code>tb_multi_symbol_top.py</code>	Multi-symbol scaling tests.
<code>tb_replay_regression.py</code>	Feeds generated ITCH; asserts book equivalence + reports strategy decisions (Phase 2).
<code>tb_phase3_pipeline.py</code>	Full-pipeline harness; reads the latency histogram over AXI-Lite (Phase 3).
<code>tb_shadow_quotes.py</code>	Feeds one quote stream to RTL <i>and</i> software; diffs decisions → reports/shadow_report.csv .
<code>tb_cl_csr.py</code>	Tests the F1 CL wrapper + CSR bridge (ITCH in / decisions out over one AXI-Lite).
<code>phase2_golden.py</code>	Pure-Python parser → book → strategy → risk runner.
<code>replay_gen.py</code> , <code>synth_itch.py</code>	Seeded ITCH generators / real-slice loader.
<code>pipeline_sva.sv</code>	SystemVerilog assertions bound to the pipeline.
<code>Makefile</code>	cocotb/Questa build (<code>SIM=questa TOPLEVEL=... MODULE=...</code>).

Host / live trading (host/)

File	What it does
strategy_sw.py	Bit-identical software mirror of strategy_imbalance.sv (kept in sync with the golden model).
risk_guard.py	Software risk checks aligned to risk_check.sv parameters.
live_feed.py	The live engine: Alpaca/IBKR quotes → strategy → risk → paper orders. Modes: <code>--alpaca</code> , <code>--symbol</code> (IBKR), <code>--simulate</code> , <code>--demo</code> ; <code>--execute</code> places real paper orders (IOC, with cancel-on-new + no-cross inventory clamp + broker-position sync).
ibkr_bridge.py	IBKR order-routing bridge (dry-run / paper).
quote_sim.py	Synthetic continuous live-quote generator (shared by <code>--simulate</code> and shadow tests).

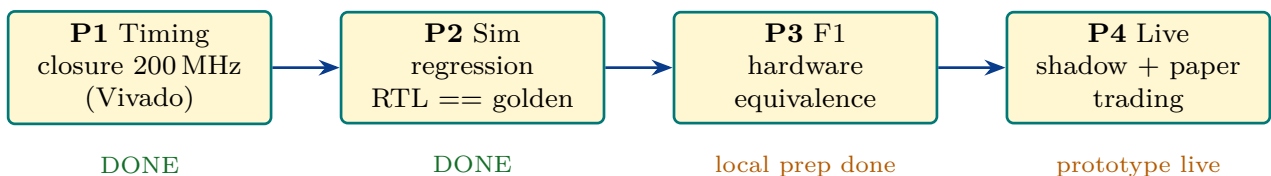
F1 deployment (f1/) & Gate-level (gatesim/)

File	What it does
cl_tick_to_trade_core.sv	Custom-Logic wrapper: CSR bridge + core behind one OCL AXI-Lite (sim-validated).
cl_tick_to_trade.sv	AWS Shell port template for the AFI build.
register_map.md	AXI-Lite address map, m_axis decision format, risk reason codes.
host_replay.py	Offline golden self-check + <code>--device</code> stub for F1 readback.
gen_netlist.tcl , compile_simlib.tcl , *_funcsim.v	Gate-level (post-synthesis) netlist + sim-lib for functional gate simulation.

Docs (docs/) & Notebooks / reports

File	What it does
part1/2/3/*.tex	Three-part learning series: foundation/architecture, RTL design & verification, integration & deployment.
implementation_results.tex	Measured Vivado results (LUTs, timing/WNS, power) for both designs.
validation_roadmap.tex	The 4-phase validation plan (timing → sim regression → F1 → live).
live_execution_issues.tex	Every real-broker issue hit in live paper trading + fixes.
why_fpga_beats_cpu_parallelism.tex	Spatial/pipeline parallelism deep dive.
executive_summary.tex , issues_log.tex	One-page summary; running issues log.
notebooks/latency_histogram*.ipynb	Latency charts (FPGA vs IBKR, per-message, histogram, score-card).
reports/shadow_report.csv , reports/soak_aapl.log	FPGA-vs-SW equivalence CSV; live-soak capture.

6. End-to-End Flow – Bench to Silicon to Live



The verification invariant (true at every phase): *feed identical input to the FPGA and to the Python golden model; assert the decisions match*. Only the input fidelity and the execution substrate change — synthetic in P1/P2, real silicon in P3, real market data in P4. This is what lets a 205 ns hardware number and a live paper trade be the *same strategy*, just realised differently.

Companion documents: [part1/2/3/*.pdf](#) (tutorial series), [implementation_results.pdf](#) (measured timing/area/power), [validation_roadmap.pdf](#) (phase plan), [live_execution_issues.pdf](#) (live-trading issues), [why_fpga_beats_cpu_parallelism.pdf](#).
Build: `pdflatex project_complete_flow.tex`.